Les architectures logicielles

Introduction

Les architectures logicielles constituent un pilier indispensable de l'ingénierie logicielle moderne. Elles permettent de construire des systèmes robustes, évolutifs et maintenables, capables d'absorber la complexité croissante des besoins métier et des contraintes techniques.

L'architecture logicielle décrit la **structure de haut niveau** d'un système : ses composants, leurs responsabilités et les interactions qui les relient. C'est le plan directeur qui guide toute décision de développement.

Principe et objectif

Principe fondamental : Diviser pour mieux régner

On découpe le système en parties : indépendantes , organisées et communicantes.

Les objectifs majeurs :

- **Maîtriser la complexité :** Décomposer un système massif en blocs gérables.
- **Garantir les attributs de qualité :** Assurer que le système est performant, sécurisé et disponible (exigences non-fonctionnelles).
- **Servir de guide :** Fournir une vision commune aux développeurs et aux parties prenantes.

Architecture vs. Conception

L'architecture et la conception sont deux activités complémentaires mais distinctes. Les confondre conduit soit à de la sur-ingénierie, soit à un manque de vision d'ensemble,

L'architecture sans conception reste abstraite ; la conception sans architecture reste anarchique.

Architecture vs. Conception

L'architecture a une portée stratégique, tandis que le design est plus tactique :

- **L'architecture est terminée** lorsque chaque acteur du projet sait **quoi faire** (aspects fonctionnels) et **comment le faire** (aspects non fonctionnels, environnement PESTEL) dans le respect des contraintes de coût, qualité et délai.
- La **conception détaillée** peut continuer à évoluer tant que les interfaces et les règles structurelles fixées par l'architecture sont respectées

Architecture vs. Conception

Dimension	Architecture	Conception
Niveau	Macroscopique — vue d'ensemble	Microscopique — vue détaillée
Question clé	Quoi construire et comment organiser ?	Comment réaliser chaque composant ?
Acteur principal	Architecte logiciel	Développeur senior / Tech Lead
Portée	Tout le système	Un module ou une fonctionnalité
Décisions types	Base de données, protocoles, découpage	Classes, méthodes, algorithmes, patterns
Livrables	Diagrammes C4, schémas déploiement	UML classes, séquences, pseudo-code
Coût de révision	Très élevé (refonte partielle/totale)	Modéré (refactoring localisé)
Horizon temporel	Long terme (années)	Court terme (sprints)

Les modèles d'architectures

Un modèle peut combiner plusieurs styles

Un modèle d'architecture (ou architecture pattern) est une **solution concrète et déployable** qui s'applique à un contexte précis avec des choix technologiques réels. Il peut combiner plusieurs styles et intègre des décisions opérationnelles (comment déployer, comment communiquer, comment scaler).

Le modèle est le **cadre technique (framework)** ou la spécification précise qui permet d'implémenter un style. C'est la "boîte à outils" réelle que les développeurs vont installer et utiliser.

Les modèles d'architectures

Pour comprendre un système complexe, il est impossible de tout regarder en même temps. C'est pourquoi on utilise des **modèles d'architectures** : ce sont des abstractions qui permettent de visualiser le logiciel sous différents angles, appelés « vues ».

Le modèle le plus célèbre est le **Modèle 4+1 vues** de Philippe Kruchten. Il part du principe qu'une seule description ne suffit pas pour satisfaire tous les acteurs (développeurs, clients, administrateurs réseau).

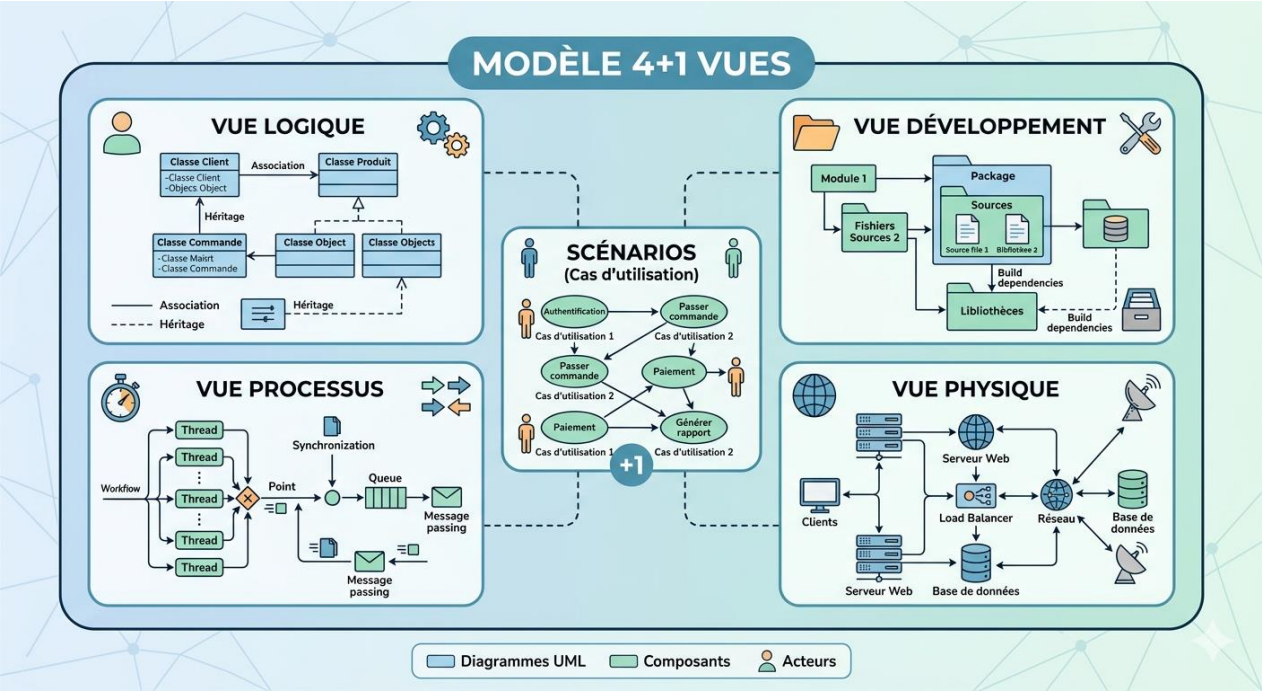
Ce modèle structure la vision d'un système : **Vue Logique, Vue Processus, Vue de Développement, Vue Physique et Les Scénarios (+1)**

Les modèles d'architectures

le **Modèle 4+1 vues** proposé par **Philippe Kruchten** donne une vision complète du système selon différents points de vue

- **Vue Logique** : Elle se concentre sur les fonctionnalités offertes à l'utilisateur final (souvent représentée par des diagrammes de classes ou d'objets).
- **Vue Processus** : Elle décrit le comportement du système au moment de l'exécution (flux de données, parallélisme, performance).
- **Vue de Développement** : Elle montre l'organisation réelle du code en modules, bibliothèques et sous-systèmes.
- **Vue Physique** : Elle schématise l'installation du logiciel sur le matériel (serveurs, réseaux, bases de données).
- **Les Scénarios (+1)** : Ce sont les cas d'utilisation qui relient toutes les autres vues entre elles pour vérifier que l'architecture répond bien aux besoins.

Les modèles d'architectures



Les modèles d'architectures

Il existe d'autres modèles qui complètent ou modernisent l'approche de Kruchten.

Certains se concentrent sur l'entreprise globale, tandis que d'autres cherchent à simplifier la documentation technique pour les développeurs d'aujourd'hui.

Les alternatives majeures : **Le Modèle C4, Le Cadre de Zachman et le modèle de Rozanski & Woods.**

Les modèles d'architectures

- **Le Modèle C4 (Context, Containers, Components, Code)** : Créé par Simon Brown, ce modèle est très populaire car il utilise une approche hiérarchique, un peu comme une carte géographique. On commence par la vue d'ensemble du système pour "zoomer" progressivement jusqu'au code.
- **Le Cadre de Zachman (Zachman Framework)** : C'est l'un des modèles les plus anciens et les plus complets pour l'architecture d'entreprise. Il utilise une grille croisant des perspectives (le décideur, le concepteur, le constructeur) avec des questions fondamentales (Quoi, Comment, Où, Qui, Quand, Pourquoi).
- **Le modèle de Rozanski & Woods** : Ce modèle s'inspire du 4+1 mais ajoute des "perspectives" transversales indispensables aujourd'hui, comme la sécurité, la performance, l'accessibilité ou l'internationalisation.

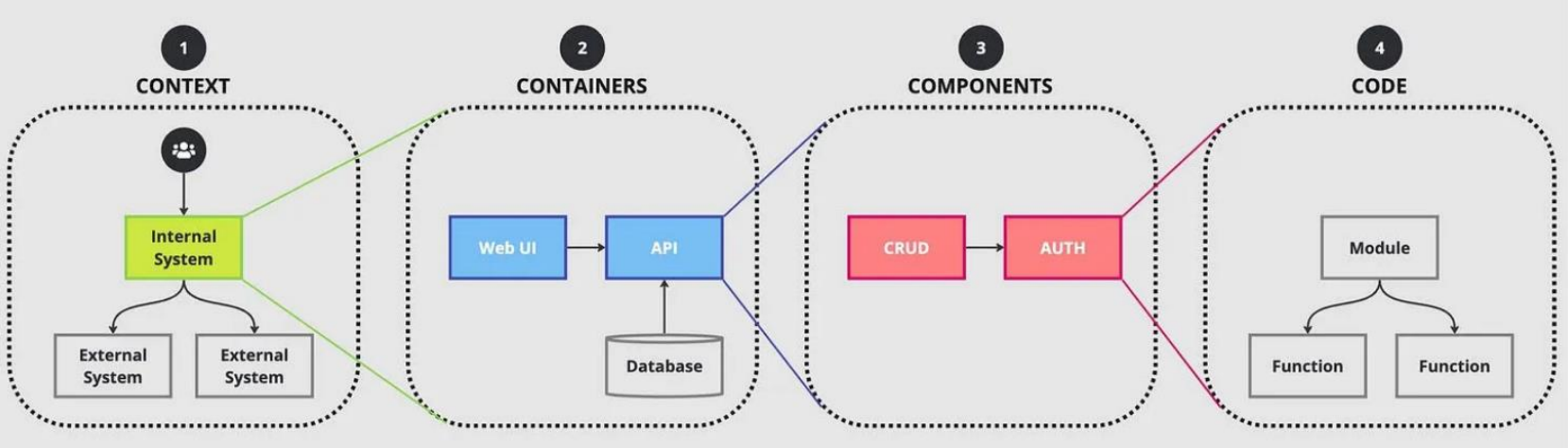
Les modèles d'architectures

Le modèle C4 — quatre niveaux de zoom

Ce modèle propose une façon structurée de documenter une architecture à travers quatre niveaux d'abstraction progressifs, chacun destiné à un public différent.

L'idée de Simon Brown est de ne pas s'encombrer de détails techniques inutiles lorsqu'on discute avec des clients (Niveau 1), tout en étant très précis pour les développeurs (Niveaux 3 et 4).

Les modèles d'architectures



Niveau	Nom	Contenu	Audience	Outil typique
1	Contexte	Système + acteurs + systèmes externes	Direction, métier, tous	Rectangle simple
2	Conteneur	Applications, bases de données, services	Architectes, tech leads	Boîtes et flèches
3	Composant	Modules internes d'un conteneur	Développeurs	Diagrammes UML
4	Code	Classes, interfaces, fonctions	Développeurs	UML classes / IDE

Les modèles d'architectures

Le cadre de Zachman

Contrairement au modèle C4 qui se focalise principalement sur le logiciel, Zachman s'attaque à l'**architecture d'entreprise** dans sa globalité.

Le cadre de Zachman est un modèle de référence utilisé en architecture des systèmes d'information.

C'est une ontologie (un système de classification) représentée sous la forme d'une **matrice 6x6**.

L'idée de John Zachman est de croiser deux dimensions pour obtenir une vue exhaustive d'une organisation : **Les Perspectives au niveau des lignes** et **Les Questions Fondamentales au niveau des colonnes**

Chaque cellule de la matrice est un "artefact" unique.

Les modèles d'architectures

- **Les Perspectives au niveau des lignes** :

Elles représentent les points de vue des différents acteurs.

1. Planificateur : Vision globale (contexte)
2. Propriétaire : Besoins métier
3. Concepteur : Modèle logique
4. Constructeur :Modèle physique
5. Sous-traitant : Implémentation technique
6. Utilisateur final : Fonctionnement réel

- **Les Questions Fondamentales au niveau des colonnes**:

Ce sont les interrogations universelles :

1. **Quoi** _ What (les données),
2. **Comment**_ How (Fonctions / processus),
3. **Où**_ Where (Localisation / réseau),
4. **Qui**_ Who (Acteurs / utilisateurs),
5. **Quand**_ When (temps / événements) et
6. **Pourquoi** _Why (la motivation, Objectifs / règles).

En croisant ces lignes avec les colonnes, on s'assure qu'on a répondu à tout.

The Zachman Framework for Enterprise Architecture™

The Enterprise Ontology™

Version 3.0

Classification Names / Audience Perspectives	What	How	Where	Who	When	Why	Classification Names / Model Names
	Composite Integrations →		← Alignment →		← Composite Integrations		
Executive Perspective (Business Content Planners)	Inventory Identification List: Inventory Types	Process Identification List: Process Types	Distribution Identification List: Distribution Types	Responsibility Identification List: Responsibility Types	Timing Identification List: Timing Types	Motivation Identification List: Motivation Types	Scope Contexts (Scope Identification Lists)
Business Mgmt Perspective (Business Concept Owners)	Inventory Definition Business Entity Business Relationship	Process Definition Business Transform Business Input/Output	Distribution Definition Business Location Business Connection	Responsibility Definition Business Role Business Work Product	Timing Definition Business Interval Business Moment	Motivation Definition Business End Business Means	Business Concepts (Business Definition Models)
Architect Perspective (Business Logic Designers)	Inventory Representation System Entity System Relationship	Process Representation System Transform System Input /Output	Distribution Representation System Location System Connection	Responsibility Representation System Role System Work Product	Timing Representation System Interval System Moment	Motivation Representation System End System Means	System Logic (System Representation Models)
Engineer Perspective (Business Physics Builders)	Inventory Specification Technology Entity Technology Relationship	Process Specification Technology Transform Technology Input /Output	Distribution Specification Technology Location Technology Connection	Responsibility Specification Technology Role Technology Work Product	Timing Specification Technology Interval Technology Moment	Motivation Specification Technology End Technology Means	Technology Physics (Technology Specification Models)
Technician Perspective (Business Component Implementers)	Inventory Configuration Tool Entity Tool Relationship	Process Configuration Tool Transform Tool Input /Output	Distribution Configuration Tool Location Tool Connection	Responsibility Configuration Tool Role Tool Work Product	Timing Configuration Tool Interval Tool Moment	Motivation Configuration Tool End Tool Means	Tool Components (Tool Configuration Models)
Enterprise Perspective (Users) The Enterprise	Inventory Instantiations Operations Entities Operations Relationships	Process Instantiations Operations Transforms Operations In/Outputs	Distribution Instantiations Operations Locations Operations Connections	Responsibility Instantiations Operations Roles Operations Work Products	Timing Instantiations Operations Intervals Operations Moments	Motivation Instantiations Operations Ends Operations Means	Operations Instances (Implementations) The Enterprise
	Composite Integrations →		← Alignment →		← Composite Integrations		
Audience Perspectives / Enterprise Names	Inventory Sets	Process Flows	Distribution Networks	Responsibility Assignments	Timing Cycles	Motivation Intentions	

*Horizontal integration lines are shown for example purposes only and are not a complete set. Composite, integrative relationships connecting every cell horizontally potentially exist.

© 1987-2011 John A. Zachman, all rights reserved. Zachman® and Zachman International® are registered trademarks of John A. Zachman
To request Permission Use of Copyright, please contact: Zachman.com

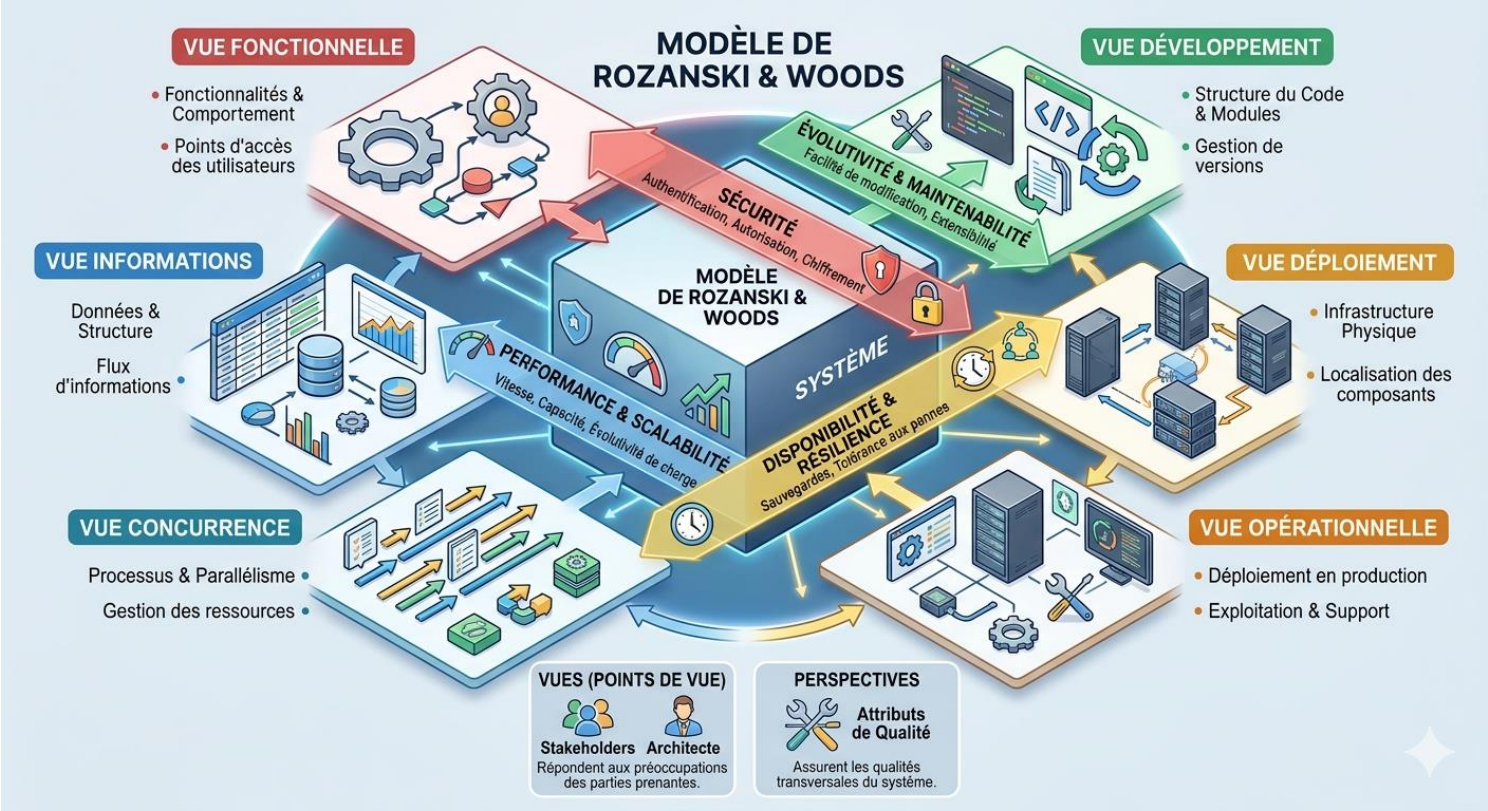
Les modèles d'architectures

Le modèle de Rozanski & Woods

Le modèle de **Rozanski et Woods** est une évolution majeure du modèle 4+1 vues. Il a été conçu pour répondre aux défis des systèmes informatiques modernes, où les contraintes comme la **sécurité**, la **disponibilité** et la **performance** sont tout aussi importantes que les fonctionnalités elles-mêmes.

Cette approche repose sur deux piliers principaux : les **Vues** (ce qu'on construit) et les **Perspectives** (comment on le construit bien).

Les modèles d'architectures



Les modèles d'architectures

Les Vues Architecturales (Architectural Views)

Contrairement à Kruchten, ils proposent un ensemble de vues plus adapté aux systèmes d'entreprise actuels :

- **Vue Fonctionnelle** : Les capacités du système (proche de la vue logique).
- **Vue Informationnelle** : Comment les données sont stockées, gérées et distribuées.
- **Vue Concurrence** : Comment les processus s'exécutent en parallèle et gèrent les ressources.
- **Vue Développement** : L'organisation du code et des tests.
- **Vue Déploiement** : L'infrastructure réseau et matérielle.
- **Vue Opérationnelle** : Comment le système est installé, administré et supporté une fois en production.

Les modèles d'architectures

Les Perspectives Architecturales (Architectural Perspectives)

C'est ici que réside la grande innovation. Une **perspective** est une préoccupation transversale qui s'applique à *toutes* les vues. Imaginez cela comme des "filtres" ou des "lunettes" que l'architecte doit porter pour vérifier la qualité du système.

Quelques exemples de perspectives clés :

- **Sécurité** : Le système est-il protégé contre les menaces à chaque niveau ?
- **Performance et Scalabilité** : Peut-il supporter une charge croissante ?
- **Disponibilité et Résilience** : Que se passe-t-il si un serveur tombe en panne ?
- **Évolution** : Est-il facile de modifier le système dans deux ans ?

Le choix d'un modèle d'architecture

Le choix d'un modèle d'architecture dépend de plusieurs facteurs : la complexité technique, la taille de l'équipe et les parties prenantes impliquées.

Trois axes aident à déterminer comment calibrer ce choix :

- **Le modèle C4 pour l'agilité** : Idéal pour les petits et moyens projets. Il permet de rester proche du code tout en offrant une vue d'ensemble claire pour l'équipe technique sans lourdeur administrative.
- **Le modèle 4+1 pour les systèmes critiques**: Recommandé quand le projet implique des enjeux de déploiement physique ou de processus parallèles complexes, car il force à regarder le système sous des angles très différents.
- **Zachman ou Rozanski & Woods pour l'échelle industrielle**: Indispensable pour les très grands projets d'entreprise ou les systèmes où la sécurité et la conformité sont des priorités absolues.

L'objectif de ces modèles est toujours de rendre le système compréhensible pour les différentes parties prenantes.